

Session 6 — Contribuer à un projet open source : fondamentaux

Objectifs de la séance

- Évaluer un projet avant d'y contribuer.
- Identifier les différents types de contributions possibles.
- Naviguer dans la structure d'un projet open source.
- Utiliser le workflow **fork** → **branch** → **PR**.

Préalable : quelques précisions

Avant de plonger dans les détails, trois rappels à garder en tête :

1. **Il n'y a pas que GitHub** — d'autres forges existent.
2. **Il n'y a pas que Git** — d'autres systèmes de versionnement existent.
3. **Tous les projets sont différents** — il faut s'adapter à chaque contexte.

Cette séance utilise GitHub et Git comme exemples car ils dominent l'écosystème actuel, mais les concepts sont transposables.

Les forges logicielles

Une **forge** est une plateforme collaborative pour le développement logiciel.

Forge	Particularité
GitHub	La plus populaire, rachetée par Microsoft (2018), propriétaire
GitLab	Open core, auto-hébergeable, CI/CD intégré
Codeberg	Non-profit (Allemagne), basée sur Forgejo
SourceHut	Minimaliste, workflow par email
Bitbucket	Atlassian (Jira, Confluence)
Gitea / Forgejo	Léger, auto-hébergeable

Fonctionnalités : hébergement des dépôts, gestion des branches et tags, historique, releases, issues, pull/merge requests, wiki, CI/CD, profils sociaux.

Au-delà de Git

Git domine, mais d'autres systèmes existent et méritent d'être connus :

Système	Caractéristique
Mercurial (hg)	Similaire à Git, utilisé par Facebook (historiquement)
Subversion (SVN)	Centralisé, encore utilisé en entreprise
Fossil	Inclut wiki, tickets, forum (SQLite l'utilise)
Pijul	Basé sur la théorie des patches, gestion des conflits innovante
Jujutsu (jj)	Compatible Git, undo/redo natif, développé par Google

Le workflow (fork → PR) reste conceptuellement similaire ; seules les commandes changent.

Chaque projet est unique

Ce qui varie d'un projet à l'autre : **taille** (1 personne à plusieurs milliers), **gouvernance** (BDFL, comité, fondation — cf. session 8), **outils** (GitHub, mailing-lists, IRC), **processus** (PRs ou patches par email), **standards de code**, **langue de communication**.

Conséquence pratique : toujours lire la documentation, observer avant de contribuer, s'adapter. Ne pas imposer ses habitudes. Un projet Linux kernel ≠ un projet npm de 100 lignes.

Partie 1 — Pourquoi contribuer ?

Motivations personnelles

Pour vous : apprendre de nouveaux langages et outils ; travailler sur du code de qualité ; construire un portfolio public ; développer son réseau ; résoudre un problème qu'on rencontre soi-même.

Pour votre carrière : GitHub comme « CV technique » ; visibilité dans la communauté ; recrutement direct par les mainteneurs ou leurs employeurs ; preuve tangible de ses compétences.

Recherche académique

Une étude de Gerosa et al. (2021) identifie **10 catégories** de motivations :

Catégorie	Description
Intrinsèque	Plaisir, challenge intellectuel
Altruisme	Aider la communauté
Réputation	Reconnaissance par les pairs
Carrière	Opportunités professionnelles
Apprentissage	Acquérir des compétences
Utilité	Résoudre son propre problème
Sociale	Appartenance à une communauté
Idéologique	Valeurs du libre
Rémunération	Payé pour contribuer
Fun	Gamification, badges

Source : *The Shifting Sands of Motivation*, Gerosa et al., 2021 — <https://arxiv.org/abs/2101.10291>.

Partie 2 — Évaluer un projet

Tous les projets ne sont pas « contribuable ». Avant de commencer, vérifier :

1. **Activité** — le projet est-il vivant ?
2. **Communauté** — y a-t-il d'autres contributeurs ?
3. **Accueil** — le projet accueille-t-il les nouveaux ?
4. **Documentation** — peut-on comprendre comment contribuer ?
5. **Réactivité** — les PRs sont-elles traitées ?

Indicateurs de santé

Bons signes : commits récents (< 1 mois) ; issues traitées régulièrement ; PRs mergées en temps raisonnable ; CONTRIBUTING.md à jour ; labels « good first issue » ; communication active et respectueuse.

Mauvais signes : dernier commit remontant à plus d'un an ; centaines d'issues ouvertes sans réponse ; PRs en attente depuis des mois ; mainteneur unique silencieux ; pas de documentation ; conflits personnels non résolus visibles dans les fils de discussion.

Métriques à vérifier

Métrique	Où la trouver	Seuil indicatif
Dernier commit	Page principale	< 3 mois

Métrique	Où la trouver	Seuil indicatif
Contributors actifs	Insights > Contributors	> 2-3 actifs récemment
Issues ouvertes	Tab Issues	Ratio fermées/ouvertes > 1
PRs ouvertes	Tab Pull requests	< 50 en attente
Temps de merge	PRs fermées	< 2 semaines médian
Stars	Page principale	Pas de seuil, contexte

Le README comme porte d'entrée

Un bon README contient : description du projet, installation / *quick start*, usage basique, comment contribuer (ou lien vers CONTRIBUTING.md), licence. Un README pauvre signale souvent un projet **pas prêt pour les contributions externes**.

Partie 3 — Types de contributions

Ce n'est pas que du code

Contributions techniques : correction de bugs, nouvelles fonctionnalités, tests, refactoring, optimisations de performance.

Contributions non techniques : documentation, traduction, design / UX, triage d'issues, réponse aux questions (forum, chat), organisation d'événements.

■ On estime que **80 % des contributions open source ne sont pas du code**.

Par où commencer

Pour débiter, privilégier les contributions à faible risque et forte valeur d'apprentissage :

1. **Typos et documentation** — risque faible, impact visible.
2. **Good first issues** — bugs simples, bien documentés par les mainteneurs.
3. **Triage** — reproduire des bugs signalés, ajouter des informations.
4. **Tests** — augmenter la couverture.
5. **Traduction** — si on parle une langue pour laquelle il manque une traduction.

Trouver des *good first issues*

Sur GitHub, chercher les labels `good first issue`, `help wanted`, `beginner friendly`.

Agrégateurs utiles :

- <https://goodfirstissue.dev>
- <https://up-for-grabs.net>
- <https://firstcontributions.github.io>
- <https://github.com/MunGell/awesome-for-beginners>

Partie 4 — Structure d'un projet

Fichiers standards

Fichier	Rôle
README.md	Présentation, installation, usage
LICENSE	Licence du projet
CONTRIBUTING.md	Guide de contribution
CODE_OF_CONDUCT.md	Règles de comportement

Fichier	Rôle
CHANGELOG.md	Historique des versions
.github/ISSUE_TEMPLATE/	Gabarits pour les issues
.github/PULL_REQUEST_TEMPLATE.md	Gabarit pour les PRs

CONTRIBUTING.md

Ce fichier explique **comment** contribuer. Contenu typique : comment signaler un bug, comment proposer une fonctionnalité, comment soumettre une PR, standards de code, processus de revue, DCO/CLA si applicable.

■ Toujours le lire avant de contribuer.

CODE_OF_CONDUCT.md

Définit les comportements acceptables dans la communauté. Standard le plus répandu :

Contributor Covenant. Autres : Citizen Code of Conduct, Ubuntu Code of Conduct. Objectifs : créer un environnement inclusif, prévenir les comportements toxiques, définir les recours en cas de problème.

Partie 5 — Le workflow de contribution

Avant de coder : « réserver » l'issue

■ Règle importante : ne jamais commencer à coder sans prévenir.

Pourquoi : éviter le travail en double, montrer son intérêt, obtenir des conseils avant de commencer.

Comment :

1. Vérifier que personne n'y travaille déjà (commentaires, *assignee*).
2. Commenter : « *I'd like to work on this, is it still available ?* »
3. Attendre confirmation (ou démarrer si pas de réponse après quelques jours).

Vue d'ensemble

1. Fork → Copier le dépôt sur votre compte
2. Clone → Télécharger localement
3. Branch → Créer une branche pour votre travail
4. Code → Modifications
5. Commit → Enregistrer vos changements
6. Push → Envoyer sur votre fork
7. PR → Proposer vos changements au projet
8. Review → Répondre aux commentaires
9. Merge → Vos changements sont intégrés

Étapes 1-2 : Fork et Clone

Sur GitHub, **Fork** via le bouton en haut à droite (crée une copie sur votre compte). En local :

```
git clone git@github.com:VOTRE-USERNAME/projet.git
cd projet
git remote add upstream git@github.com:ORIGINAL/projet.git
```

Convention : origin pointe sur votre fork, upstream sur le dépôt original.

Étape 3 : Créer une branche

Toujours travailler sur une branche dédiée, jamais sur main.

```
git checkout main
git pull upstream main
```

```
git checkout -b fix/typo-in-readme
```

Conventions de nommage typiques : `fix/...`, `feature/...`, `docs/...`, `refactor/...`

Étapes 4-5 : Code et Commit

Règle d'or : **commits atomiques** — un commit = un changement logique.

```
git status
git diff
git add fichier_modifie.py
git commit -m "Fix typo in installation instructions"
```

Message de commit clair : première ligne < 50 caractères (résumé), ligne vide, corps explicatif si nécessaire. Détails approfondis en session 7.

Étapes 6-7 : Push et Pull Request

```
git push origin fix/typo-in-readme
```

Sur GitHub : aller sur son fork, cliquer *Compare & pull request*, remplir le gabarit, expliquer le changement, soumettre.

Étapes 8-9 : Review et Merge

Ce qui peut arriver :

Feedback positif : « LGTM » (*Looks Good To Me*), merge rapide, remerciements.

Demandes de changements : suggestions de style, demande de tests supplémentaires, questions sur l'approche, refus motivé.

Patience : les mainteneurs sont souvent bénévoles, le temps de réponse peut être long.

Garder son fork à jour

Votre fork diverge du projet original au fil du temps. Synchroniser régulièrement :

```
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

Synchroniser **avant** de créer une nouvelle branche. Le rebase et la gestion des conflits sont traités en session 7.

Erreurs à éviter (résumé)

- Commencer à coder sans discuter (sur les grosses features, toujours discuter d'abord).
- PR massive, inreviewable, qui mélange plusieurs sujets.
- Ignorer le `CONTRIBUTING.md`.
- Disparaître après avoir soumis la PR.
- Prendre les critiques personnellement.

Ces thèmes sont repris et approfondis en **session 7**.

Ce qu'il faut retenir

1. **Évaluer** le projet avant de contribuer (activité, accueil, documentation).
2. **Commencer petit** : typos, documentation, *good first issues*.
3. **Lire** `CONTRIBUTING.md` et `CODE_OF_CONDUCT.md` avant toute chose.

4. **Workflow** : Fork → Branch → Code → PR.
5. **Patience et respect** envers les mainteneurs bénévoles.

Pour aller plus loin

- *First Contributions* : <https://github.com/firstcontributions/first-contributions>
- *Good First Issue* : <https://goodfirstissue.dev>
- *How to Contribute to Open Source* (GitHub) : <https://opensource.guide/how-to-contribute/>
- Karl Fogel, *Producing Open Source Software* : <https://producingoss.com> — référence complète, chapitre « Getting Started ».
- Guide Mozilla : <https://mozilla.github.io/open-leadership-training-series/>
- Open Source Masterclass (MOOC) : <https://opensourcemasterclass.org/>